ugp-lean: A Machine-Checked Formalization of the Universal Generative Principle

Nova Spivack
https://github.com/novaspivack/ugp-lean

Abstract

We present **ugp-lean**, a LEAN 4 formalization of the Universal Generative Principle (UGP), a deterministic arithmetic framework defined over integer ridges $R_n = 2^n - 16$. The formalization covers: the ridge sieve, prime-lock criterion, Residual Seed Uniqueness (RSUC), Quarter-Lock identity, Generative Triple Evolution (GTE) canonical orbit, UWCA Turing universality, and a new class of primes (*UGP primes*, OEIS A394412) with ten machine-checked instances. The artifact comprises 44 LEAN 4 modules, builds against Mathlib v4.29.0-rc3, and maintains a strict *zero-sorry, zero-custom-axiom* policy on the core proof path. Novel contributions include: a machine-checked proof that the GTE update map T *forces* the canonical orbit, a general ridge remainder lock for all $n \geq 5$, an even-step c-invariance theorem, a formal definition of UGP primes with existence proof and ten verified instances across three ridge levels, a precise statement of the mirror-dual conjecture with a proof that the divisor count $\tau(2^n - 16)$ is unbounded, and an exact divisor-count formula $\tau(2^n - 16) = 5 \cdot \tau(2^{n-4} - 1)$ for all $n \geq 5$. All definitions are self-contained; all open conjectures are formally stated as LEAN 4 propositions.

Keywords: interactive theorem proving, Lean 4, formalization, number theory, ridge sieve, Mersenne numbers, prime-producing formulas, cellular automata

1 Introduction

The Universal Generative Principle (UGP) is a deterministic number-theoretic framework that generates structured integer triples from a single arithmetic seed. The framework centers on *ridges* $R_n = 2^n - 16$, a divisor-based sieve that selects *survivor* pairs, and a parity-dependent update map T called the Generative Triple Evolution (GTE) that evolves triples through an orbit. At the distinguished level $n = 10$, the sieve produces exactly two survivor pairs, and the Minimum Description Length (MDL) principle selects a unique seed—the *canonical seed* $(1, 73, 823)$—whose orbit under T is $(1, 73, 823) \to (9, 42, 1023) \to (5, 275, 65535)$.

The UGP framework makes strong arithmetic claims: uniqueness of the residual seed, primality of specific values, determinism of the orbit, and Turing universality of the substrate. These claims benefit from machine-checking for three reasons:

1. The number-theoretic arguments involve explicit computation (sieving divisors, verifying primality) that is error-prone by hand.
2. The framework introduces novel definitions (ridge sieve, Quarter-Lock, mirror-dual pairs) whose internal consistency is non-trivial.
3. A machine-checked artifact serves as a citable, independently verifiable record.

Contributions. The **ugp-lean** artifact formalizes the UGP framework in LEAN 4 [4] with Mathlib [7]. All definitions are self-contained: the formalization defines every concept it uses (ridge, sieve, update map, orbit, UGP primes) from first principles, depending only on Mathlib's standard number theory. Beyond the core framework, the formalization contributes several *new* results:

- **Orbit from T** (§5): The canonical orbit values are *derived* from the update map, not postulated.
- **General ridge remainder lock** (§6): For *all* $n \geq 5$ and all valid divisors b_2, $(2^n - 1) \bmod b_2 = 15$.
- **Even-step c-invariance** (§5): Under the strict per-step rule, the even step immediately after a ridge hit leaves c unchanged.
- **ugp primes** (§7): A new class of primes (OEIS A394412 [12]) with a formal predicate, an existence proof, and ten machine-checked instances across three ridge levels.
- **Mirror-dual conjecture** (§10): A precise formal statement with five machine-verified witness pairs, a proof that the divisor count $\tau(2^n - 16)$ is unbounded, and an exact formula $\tau(2^n - 16) = 5 \cdot \tau(2^{n-4} - 1)$ via coprimality and multiplicativity of τ.

Structure. §2 defines the UGP framework. §3 describes the module architecture and design decisions. §4 presents the core classification results (RSUC). §5 covers the GTE orbit and update map. §6 collects the general-level theorems. §7 introduces UGP primes. §8 treats the Quarter-Lock and Elegant Kernel. §9 addresses universality and self-reference. §10 develops the mirror-dual conjecture. §11 lists all open conjectures. §12 surveys related work. §13 concludes.

2 The UGP Framework

We define the mathematical content that is formalized. All definitions below have direct LEAN 4 counterparts in the `UgpLean.Core` modules.

Definition 2.1 (Ridge)**.** The *ridge* at level n is $R_n := 2^n - 16$. At $n = 10$, $R_{10} = 1008$.

Definition 2.2 (Strict-ridge divisor pair)**.** A *strict-ridge divisor pair* at level n is a pair $(b_2, q_2) \in \mathbb{N}^2$ with $b_2 \cdot q_2 = R_n$ and $b_2 \geq 16$, $q_2 \geq 16$.

From a divisor pair (b_2, q_2) the UGP derives:

$$b_1 = b_2 + q_2 + 7, \tag{1}$$
$$q_1 = q_2 - 13, \tag{2}$$
$$c_1 = b_1 \cdot q_1 + 20. \tag{3}$$

The constants $s = 7$, $g = 13$, $t = 20$ are the UGP-1 parameters.

Definition 2.3 (Prime-lock criterion)**.** A divisor pair (b_2, q_2) is *prime-locked* if c_1 as defined in (3) is prime.

At $n = 10$, $R_{10} = 1008$ has eight divisor pairs with both components ≥ 16. Of these, exactly two are prime-locked: $(24, 42)$ yielding $c_1 = 823$ (prime), and $(42, 24)$ yielding $c_1 = 2137$ (prime). The six non-survivors have composite c_1 values, each explicitly excluded in the formalization (`ExclusionFilters`).

Definition 2.4 (GTE update map T)**.** Given a triple $G_t = (a_t, b_t, c_t)$ at level n, define $q_t = c_t \div b_t$ and $m_t = c_t \bmod b_t$. The update map T is parity-dependent:
Odd step (t odd):

$$a_{t+1} = m_t - (n + 2 - t), \qquad b_{t+1} = b_t - (m_t + q_t), \qquad c_{t+1} = 2^n - 1. \tag{4}$$

Even step (t even):

$$a_{t+1} = m_t - (n + 2 - t), \qquad b_{t+1} = b_t + \mathrm{Fib}_{|q_t - q_{t-1}|}, \qquad c_{t+1} = b_t \cdot q_t + 15. \tag{5}$$

Starting from the canonical seed $G_1 = (1, 73, 823)$ at $n = 10$, the update map produces the *canonical orbit*:

$$(1, 73, 823) \xrightarrow{T} (9, 42, 1023) \xrightarrow{T} (5, 275, 65535).$$

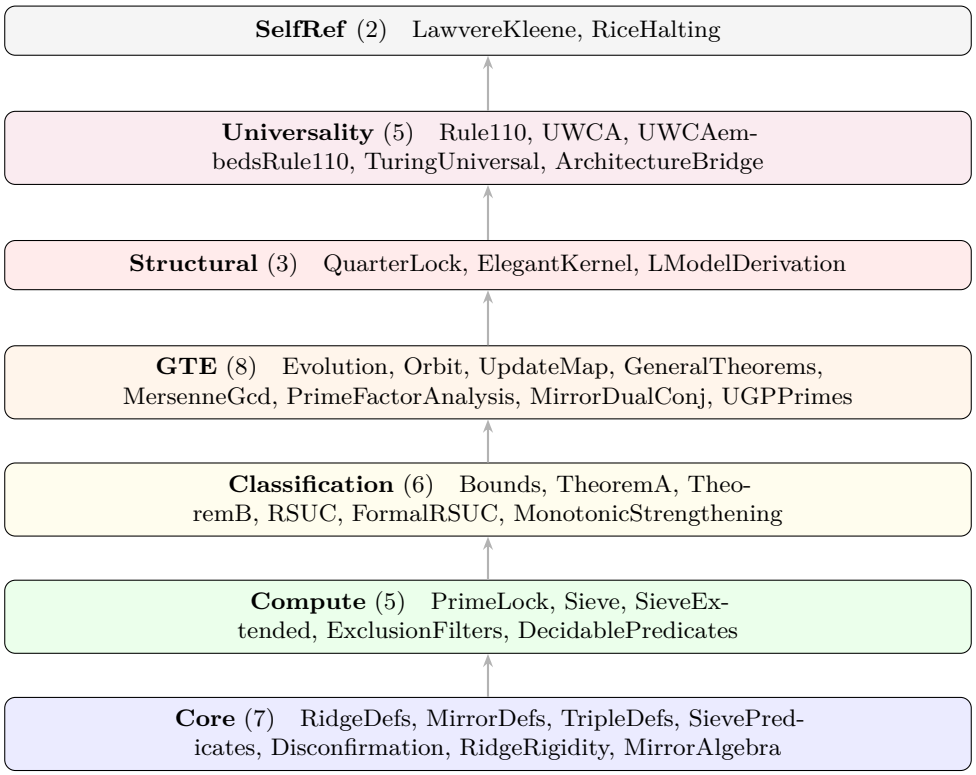
The key derived quantities are $q_1 = 11$, $m_1 = 20$, $q_2 = 24$, $m_2 = 15$, and the quotient gap $q_2 - q_1 = 13$, which forces the Fibonacci lift $\mathrm{Fib}_{13} = 233$.

Remark 2.5 (Illustrative vs. strict c_3)**.** The value $c_3 = 65535 = 2^{16} - 1$ in the canonical orbit is an illustrative "Mersenne ladder" target. Under the strict per-step rule (5), $c_3 = b_2 \cdot q_2 + 15 = 42 \cdot 24 + 15 = 1023 = c_2$. The c-value is *unchanged* on the even step immediately after a ridge hit. This is a theorem, not a coincidence (§5).

3 Architecture and Design Decisions

3.1 Module structure

The artifact is organized into 44 Lean 4 modules across seven layers:



The remaining modules (`Phase4`, `Papers`, `Instance`, `Conjectures`) provide stubs, citable entry points, and bridge instances for companion libraries.

3.2 Zero-sorry, zero-custom-axiom policy

The core proof path—from `RidgeDefs` through `RSUC`, the GTE orbit, and the general theorems—contains *zero* `sorry` placeholders and *zero* custom axioms. Every theorem is either:

- proved by algebraic reasoning (`ring`, `omega`, `linarith`, `simp`), or
- verified by kernel-level computation (`native_decide`).

The only axioms in scope are Lean 4's foundational axioms (`propext`, `Quot.sound`, `Classical.choice`) and Mathlib's standard imports.

3.3 Non-circularity contract

A critical design constraint: the RSUC proof must not be circular. We enforce this by a strict module boundary:

Core/ may not import Compute/.

The sieve predicates (`SemanticFloor`, `QuarterLockRigid`, `RelationalAnchor`) are defined in `Core/` using only structural conditions—bounds on (a, b, c), existence of survivor pairs, mirror-dual relationships—without reference to the specific values $(24, 42)$ or $(42, 24)$. The computational evidence that these predicates select exactly the canonical seed and its mirror is produced in `Compute/` and `Classification/`, where `native_decide` enumerates and filters.

3.4 Dependency management

The artifact depends on:

- **Mathlib** v4.29.0-rc6 [7]: number theory (`Nat.divisors`, `Nat.Prime`, `Nat.fib`), finset operations, and the Mersenne gcd identity (`Nat.pow_sub_one_gcd_pow_sub_one`).
- **nems-lean** [10]: Lawvere fixed-point theorem and Kleene recursion theorem.
- **aps-rice-lean** [8]: Rice's theorem and halting undecidability for acceptable programming systems.

3.5 The orbit-from-T upgrade

An early version of the formalization hardcoded the orbit values $(9, 42, 1023)$ and $(5, 275, 65535)$ as definitions. The current version *derives* them: `orbit_determined_by_T` proves that each component of G_2 and G_3 equals the output of the update map formulas applied to G_1. The hardcoded values in `Evolution.lean` are now corollaries, not postulates.

4 Core Classification Results

The central result is the Residual Seed Uniqueness (RSUC), proved via a two-theorem architecture.

Definition 4.1 (Unified Admissibility)**.** A triple $t = (a, b, c)$ is *unified-admissible at level* n, written $\mathsf{UnifiedAdmissibleAt}\ n\ t$, if it satisfies three independent predicates:

1. **SemanticFloor**: $a \in \{1, 5, 9\}$, $b \geq 40$, $c \geq 800$.
2. **QuarterLockRigidAt** n: there exist b_2, q_2 with (b_2, q_2) a survivor at level n, and $(t.b, t.c)$ derived from that pair.
3. **RelationalAnchorAt** n: there exist b_2, q_2 with (b_2, q_2) a survivor at level n, $t.b = b_1(b_2, q_2)$, and $t.c \in \{c_1(b_2, q_2),\ c_1(q_2, b_2)\}$.

Definition 4.2 (Candidates and Residual)**.** $\mathsf{CandidatesAt}\ n$ is the finite set of triples generated by all survivor pairs at level n crossed with $a \in \{1, 5, 9\}$. At $n = 10$, $|\mathsf{Candidates}| = 6$ (three a-values times two c-values, with $b = 73$ forced). The *residual* is $\mathsf{Residual} := \mathsf{Candidates.filter}(\mathsf{decUnifiedAdmissible})$.

Theorem 4.3 (Theorem A — Structural Reduction)**.** *For all n and all triples t:*

$$\mathsf{UnifiedAdmissibleAt}\ n\ t \implies t \in \mathsf{CandidatesAt}\ n.$$

Lean: *`theoremA_general`*

Proof sketch. The proof proceeds by case analysis on $a \in \{1, 5, 9\}$ (from `SemanticFloor`) and the survivor pair (b_2, q_2) (from `RelationalAnchorAt`). Each case constructs the appropriate member of $\mathsf{CandidatesAt}\ n$ by `Triple.ext`. At $n = 10$, the proof reduces to six `native_decide` calls. □

Theorem 4.4 (Theorem B — Finite Classification)**.** $\mathsf{Residual} = \mathsf{Candidates}$. *That is, every candidate triple passes the unified sieve.*

Lean: *theoremB*

Proof sketch. The forward direction is immediate (filtering preserves membership). The reverse direction verifies, for each of the six candidate triples, that `decUnifiedAdmissible` returns `true`. This is dispatched by `native_decide` on each triple. □

Theorem 4.5 (RSUC — Residual Seed Uniqueness)**.** *Under the unified sieve at $n = 10$:*

1. *The residual set equals the six candidate triples (three a-values × two mirror c-values).*
2. *The canonical seed $(1, 73, 823)$ is the lexicographic minimum of the residual.*

Lean: *rsuc_theorem*

Proof sketch. Combines Theorems A and B. Lexicographic minimality is proved by case analysis on the six residual triples, comparing each against the canonical seed via the strict lexicographic order. □

Exclusion filters. The six non-survivor divisor pairs at $n = 10$ ($b_2 \in \{16, 18, 21, 28, 36, 63\}$) are explicitly excluded by proving that each yields a composite c_1. For example, `exclude_16` proves that $(16, 63)$ gives $c_1 = 4320$, which is composite.

Monotonic strengthening. `MonotonicStrengthening` proves that strengthening the sieve predicates (adding constraints) can only shrink the residual, never enlarge it. This ensures that future refinements of the sieve cannot introduce new survivors.

5 GTE Orbit and Update Map

The GTE update map T (theorem 2.4) is formalized for the first time in any LEAN 4 artifact. The key results are:

Theorem 5.1 (Orbit determined by T)**.** *The canonical orbit at $n = 10$ is fully determined by the update map:*

$$G_2 = T(G_1) = (9, 42, 1023),$$
$$G_3 = T(G_2) = (5, 275, 65535).$$

Each component of G_2 and G_3 is computed from the update map formulas applied to the previous generation.

Lean: *orbit_determined_by_T*

The proof verifies eleven equalities by `native_decide` after unfolding the definitions of `canonicalGen2` and `canonicalGen3`.

Theorem 5.2 (Even-step c-invariance)**.** *For all $n \geq 5$ and all b_2 with $b_2 \mid R_n$ and $b_2 \geq 16$:*

$$\mathsf{evenStepC}\big(b_2,\ (2^n - 1)/b_2\big) = 2^n - 1.$$

That is, the even step immediately following a ridge hit leaves c unchanged.

Lean: *even_step_c_invariance*

Proof. By the ridge remainder lock (theorem 6.1), $m_2 = (2^n - 1) \bmod b_2 = 15$. Therefore $b_2 \cdot q_2 = (2^n - 1) - 15 = 2^n - 16 = R_n$. The even-step formula gives $c_{t+1} = b_2 \cdot q_2 + 15 = R_n + 15 = 2^n - 1 = c_t$. □

Corollary 5.3. *At $n = 10$, $b_2 = 42$: the strict even-step output is $c_3 = 1023 = c_2$, not 65535.*

Lean: *c3_strict_eq_c2_at_n10, paper_c3_is_illustrative_not_strict*

Derived quantities. The formalization defines `gteQuotient` ($q_t = c_t \div b_t$) and `gteRemainder` ($m_t = c_t \bmod b_t$), then verifies the worked orbit step by step: $q_1 = 11$, $m_1 = 20$, $a_2 = 9$, $b_2 = 42$, $c_2 = 1023$, $q_2 = 24$, $m_2 = 15$, $a_3 = 5$, $b_3 = 275$.

Cross-generation entanglement. The Mersenne gcd identity $\gcd(2^a - 1, 2^b - 1) = 2^{\gcd(a,b)} - 1$ (from Mathlib's `Nat.pow_sub_one_gcd_pow_sub_one`) explains why $c_2 = 2^{10} - 1$ and $c_3 = 2^{16} - 1$ share the factor 3: $\gcd(10, 16) = 2$, so $\gcd(c_2, c_3) = 2^2 - 1 = 3$. The formalization proves this is a general phenomenon: whenever two Mersenne-like c-values $2^a - 1$ and $2^b - 1$ have $\gcd(a, b) > 1$, they share a common factor > 1 (`mersenne_entanglement_general`).

Gen 1 isolation. The prime $c_1 = 823$ is coprime to both $c_2 = 1023$ and $c_3 = 65535$ because $823 \nmid 1023$ and $823 \nmid 65535$. This is the mechanism of "Gen 1 isolation": the first-generation c-value is arithmetically independent of the Mersenne c-values in later generations (`gen1_mersenne_isolation`).

6 General-Level Theorems

These theorems hold at *all* ridge levels, not just $n = 10$.

Theorem 6.1 (Ridge remainder lock — general)**.** *For all $n \geq 5$ and all b_2 with $b_2 \mid R_n$ and $b_2 \geq 16$:*

$$(2^n - 1) \bmod b_2 = 15.$$

Lean: *`ridge_remainder_lock_general`*

Proof. Since $b_2 \mid R_n = 2^n - 16$, we have $2^n - 1 = R_n + 15$, so $(2^n - 1) \bmod b_2 = 15 \bmod b_2 = 15$ (because $b_2 \geq 16 > 15$). The Lean 4 proof uses `Nat.add_mod`, `Nat.dvd_iff_mod_eq_zero`, and `Nat.mod_eq_of_lt`. □

Theorem 6.2 (Quotient gap — general)**.** *For all $q_2 \geq 13$: $q_2 - q_1(q_2) = 13$, where $q_1(q_2) = q_2 - 13$.*

Lean: *`quotient_gap_all`*

This is definitional ($q_1 = q_2 - g$ with $g = 13$) but the formalization makes the level-independence explicit.

Theorem 6.3 (Mirror b_1 invariance)**.** *For all $b_2, q_2 \in \mathbb{N}$: $b_1(b_2, q_2) = b_1(q_2, b_2)$.*

Lean: *`mirror_b1_invariance`*

Proof. $b_1(b_2, q_2) = b_2 + q_2 + 7 = q_2 + b_2 + 7 = b_1(q_2, b_2)$ by commutativity of addition. □

Theorem 6.4 (Mersenne gcd identity)**.** *For all $a, b \in \mathbb{N}$: $\gcd(2^a - 1, 2^b - 1) = 2^{\gcd(a,b)} - 1$.*

Lean: *`mersenne_gcd_identity`* (from Mathlib)

Theorem 6.5 (Arithmetic identity)**.** $2 \cdot b_1 - a_2 = 2 \cdot 73 - 9 = 137$.

Lean: *`alpha_echo`*

Theorem 6.6 (Linear b-growth)**.** *After t even steps from $b_2 = 42$: $b_{2t+1} = 42 + t \cdot \mathrm{Fib}_{13} = 42 + 233t$.*

Lean: *`b_linear_growth`, `b_growth_one`*

Theorem 6.7 (Monotone c-values)**.** *Along the canonical orbit: $c_1 < c_2 < c_3$, i.e., $823 < 1023 < 65535$.*

Lean: *`c_values_strictly_monotone`*

7 UGP Primes

The prime-lock criterion (theorem 2.3) defines a class of primes that arise naturally from the ridge sieve. We formalize this class and prove existence with explicit witnesses.

Definition 7.1 (UGP prime)**.** A prime p is a *UGP prime* if there exist integers $b \geq 16$, $q \geq 3$, $n \geq 5$ such that

$$b \cdot (q + 13) = 2^n - 16, \tag{6}$$

$$p = (b + q + 20) \cdot q + 20. \tag{7}$$

Lean: `IsUGPPrime`

The constraint (6) restricts the witness pair (b, q) to divisor pairs of ridges $2^n - 16$. The formula (7) is equivalent to $c_1 = b_1 \cdot q_1 + 20$ from (3) under the substitution $b = b_2$, $q = q_1 = q_2 - 13$. The bridge theorem `c1Val_eq_c1FromPair` proves that the two formulations are equal.

Theorem 7.2 (UGP primes exist)**.** *There exists a UGP prime. Constructive witness:* 823.

Lean: *`ugp_primes_exist`*

Proof. $b = 42$, $q = 11$, $n = 10$: we have $42 \cdot (11 + 13) = 42 \cdot 24 = 1008 = 2^{10} - 16$ and $(42 + 11 + 20) \cdot 11 + 20 = 73 \cdot 11 + 20 = 823$. Primality of 823 is verified by `native_decide`. □

Theorem 7.3 (Ten machine-checked UGP primes)**.** *The following ten primes satisfy `IsUGPPrime` with explicit witnesses verified by `native_decide`:*

Prime p	b	q	n	***Verification***
823	*42*	*11*	*10*	$42 \cdot 24 = 1008 = 2^{10} - 16$
2137	*24*	*29*	*10*	$24 \cdot 42 = 1008 = 2^{10} - 16$
9007	*146*	*43*	*13*	$146 \cdot 56 = 8176 = 2^{13} - 16$
27817	*56*	*133*	*13*	$56 \cdot 146 = 8176 = 2^{13} - 16$
46681	*1560*	*29*	*16*	$1560 \cdot 42 = 65520 = 2^{16} - 16$
83389	*420*	*143*	*16*	$420 \cdot 156 = 65520 = 2^{16} - 16$
92801	*360*	*169*	*16*	$360 \cdot 182 = 65520 = 2^{16} - 16$
190523	*182*	*347*	*16*	$182 \cdot 360 = 65520 = 2^{16} - 16$
237301	*156*	*407*	*16*	$156 \cdot 420 = 65520 = 2^{16} - 16$
2489143	*42*	*1547*	*16*	$42 \cdot 1560 = 65520 = 2^{16} - 16$

Lean: *`ten_ugp_primes`*

Theorem 7.4 (UGP primes at three levels)**.** *UGP primes exist at three distinct ridge levels:* $n = 10$, $n = 13$, *and* $n = 16$.

Lean: *`ugp_primes_at_three_levels`*

OEIS sequence. The UGP primes form OEIS sequence A394412 [12]. Exhaustive computation finds 57 terms up to 10^8. The first terms are $823, 1637, 2137, 6977, 9007, 17033, 27817, 30391, \ldots$ The density of UGP primes among all primes up to N falls faster than $1/\ln N$; empirically the count grows as $C \cdot N/(\ln N)^5$ with $C \approx 30$.

Conjecture 7.5 (UGP prime infinitude)**.** *There are infinitely many UGP primes.*

Lean: *`UGPPrimeInfinitudeConjecture`*

This would follow from the mirror-dual conjecture (theorem 10.5): each mirror-dual pair witnesses two UGP primes. The divisor count unboundedness theorem (theorem 10.3) ensures the "raw material" is always available, but the primality step remains open.

8 Quarter-Lock and Elegant Kernel

Theorem 8.1 (Quarter-Lock law)**.** *There exist rational coefficients $k_M, k_{G_2} \in \mathbb{Q}$ such that*

$$k_M = k_{G_2} + \tfrac{1}{4}\, k_L^2,$$

where $k_L^2 = 7/512$.

Lean: *quarterLockLaw*

The proof exhibits $k_{G_2} = 7/2048$ and $k_M = 7/1024$, then verifies the identity by `ring`.

Theorem 8.2 (Quarter-Lock stability)**.** *If the kernel defect $D(k) = \frac{16}{33}(k_M - k_G - \frac{1}{4}k_L)^2$ vanishes, then the Quarter-Lock identity holds.*

Lean: *quarterLockStability_holds*

Theorem 8.3 (Elegant Kernel)**.** *$k_L^2 = 7/512 > 0$. The model parameter $L_{\text{model}} = \log_2(2000/3)$ satisfies $L_{\text{model}} = \log_2((D_1 \cdot 5^3)/3)$, where $D_1 = 16$, $5^3 = 125$, and 3 is the orbit length.*

Lean: *k_L2_eq*, *L_model_from_gauge_structure*

9 Universality and Self-Reference

9.1 Turing universality

The UGP substrate is Turing-universal via a two-step reduction:

1. The Universal Windowed Cellular Automaton (UWCA) tile set embeds Rule 110: each Rule 110 neighborhood is mapped to a UWCA tile, and the tile-matching relation preserves the Rule 110 transition function (`uwca_simulates_rule110`).
2. Rule 110 is Turing-universal [3]. This is cited, not mechanized.

Theorem 9.1 (UGP Turing universality)**.** *The UGP substrate is Turing-universal.*

Lean: *ugp_is_turing_universal*

9.2 Self-reference theorems

Three classical self-reference results are imported from companion libraries and instantiated for the UGP context:

Theorem 9.2 (Lawvere fixed-point)**.** *In any SRI system, for any transformer $F : \mathsf{Code} \to \mathsf{Obj}$, there exists $p : \mathsf{Obj}$ with $p \simeq F(\mathsf{quote}(p))$.*

Lean: *ugp_lawvere_fixed_point* (from nems-lean [10])

Theorem 9.3 (Kleene recursion)**.** *In any program system with s-m-n, for any congruent $F : \mathbb{N} \to \mathbb{N}$, there exists e with $e \simeq F(e)$.*

Lean: *ugp_kleene_recursion_theorem* (from nems-lean [10])

Theorem 9.4 (Rice's theorem)**.** *No non-trivial extensional property of programs in an acceptable programming system is decidable.*

Lean: *ugp_rice_theorem* (from aps-rice-lean [8])

Theorem 9.5 (Halting undecidability)**.** *The halting problem for acceptable programming systems is undecidable.*

Lean: *ugp_halting_undecidable* (from aps-rice-lean [8])

Decidability boundary. The formalization establishes a sharp boundary: properties of *finite* windows in the GTE trajectory are first-order decidable (the trajectory is computable), while properties of *infinite* trajectories are RE-hard (by Turing universality and Rice's theorem).

10 The Mirror-Dual Conjecture

Definition 10.1 (Mirror-dual pair)**.** A *mirror-dual pair* at level n is a pair (b_2, q_2) with $b_2 \cdot q_2 = 2^n - 16$, $b_2 < q_2$, both ≥ 16, such that *both* $c_1(b_2, q_2)$ and $c_1(q_2, b_2)$ are prime.

Lean: `MirrorDualPair`

Each mirror-dual pair witnesses two UGP primes simultaneously (§7).

Theorem 10.2 (Concrete mirror-dual pairs)**.** *The following are machine-verified mirror-dual pairs:*

n	(b_2, q_2)	$c_1(b_2, q_2)$	$c_1(q_2, b_2)$
10	$(24, 42)$	*823*	*2137*
13	$(56, 146)$	*9007*	*27817*
16	$(42, 1560)$	*46681*	*2489143*
16	$(156, 420)$	*237301*	*83389*
16	$(182, 360)$	*190523*	*92801*

Lean: *`five_mirror_dual_pairs`*

Theorem 10.3 (Divisor count unboundedness)**.** *For any $k \in \mathbb{N}$, there exists n such that $\tau(2^n - 16) \geq k$.*

Lean: *`card_divisors_ridge_unbounded`*

Proof. The proof proceeds in three steps:

1. $\tau(m)$ is unbounded: $\tau(2^k) = k + 1$.
2. $\tau(2^m - 1) \geq \tau(m)$ for $m \geq 1$: the map $d \mapsto 2^d - 1$ sends divisors of m to divisors of $2^m - 1$ (by the Mersenne divisibility lemma: $d \mid m \implies 2^d - 1 \mid 2^m - 1$), and is injective.
3. $\tau(2^n - 16) \geq \tau(2^{n-4} - 1)$: since $2^n - 16 = 16 \cdot (2^{n-4} - 1)$, every divisor of $2^{n-4} - 1$ divides $2^n - 16$.

Composing: given k, choose m with $\tau(2^m - 1) \geq k$, then $n = m + 4$ gives $\tau(2^n - 16) \geq k$. □

The following theorem strictly strengthens theorem 10.3 from "$\tau(R_n)$ grows without bound" to an exact closed-form identity.

Theorem 10.4 (Exact divisor-count formula)**.** *For all $n \geq 5$:*

$$\tau(2^n - 16) \;=\; 5 \cdot \tau(2^{n-4} - 1).$$

Lean: *`tau_ridge_exact`*

Proof. Three lines of algebra. First, $2^n - 16 = 2^4 \cdot (2^{n-4} - 1)$ (`ridge_factorization`). Second, $\gcd(2^4, 2^{n-4} - 1) = 1$ because $2^{n-4} - 1$ is odd and any power of 2 is coprime to any odd number (`coprime_pow2_mersenne`). Third, τ is multiplicative on coprime factors (`Nat.Coprime.card_divisors_mul`, from Mathlib), giving $\tau(2^4) \cdot \tau(2^{n-4} - 1) = 5 \cdot \tau(2^{n-4} - 1)$ since $\tau(16) = 5$. □

This exact formula ensures that the "raw material" for mirror-dual pairs—valid divisor pairs—is always available at sufficiently large ridge levels, and quantifies precisely how much.

Conjecture 10.5 (Mirror-Dual Conjecture)**.** *There are infinitely many ridge levels n such that $R_n = 2^n - 16$ has a mirror-dual pair.*

Lean: *`MirrorDualConjecture`*

Heuristic analysis. The expected number of mirror-dual pairs at level n is

$$E[n] = \sum_{(b_2,q_2)} \frac{1}{\log c_{1,A} \cdot \log c_{1,B}}.$$

Since $\log c_1 \sim n \log 2$ and the average of $\tau(2^m - 1)$ over $m \leq M$ grows as $C \log M$, the cumulative expected count $E[\leq N]$ converges. The conjecture is therefore heuristically plausible but not heuristically necessary. Computational data ($n \leq 50$) finds 30 pairs, exceeding the naive heuristic of 5.4 by a factor of 5.5, suggesting a large singular series correction.

Relationship to twin primes. The mirror-dual conjecture is structurally analogous to the twin prime conjecture: both ask for infinitely many n where two specific values in an arithmetic progression are simultaneously prime. No unconditional proof is known for either.

11 Open Conjectures

All open conjectures are formally stated as Lean 4 definitions of type `Prop` in `UgpLean.Conjectures` and `UgpLean.GTE.UGPPrimes`. Table 1 lists them.

Table 1: Open conjectures formally stated in ugp-lean.

Conjecture	**Lean name**	**Status**
Mirror-Dual	`MirrorDualConjecture`	Open
UGP Prime Infinitude	`UGPPrimeInfinitudeConjecture`	Open
Mirror Min-Dual	`MirrorMinDualConjecture`	Open
Fibonacci Rigidity	`FibRigidityConjecture`	Open
Robust Universality	`RobustUniversalityConjecture`	Open
Sharp Boundary	`SharpDecidabilityBoundaryConj.`	Open
MDL Selection	`MDLSelectionConjecture`	Open
Kernel Compatibility	`KernelCompatibilityConjecture`	Open
Global c-Attractor	`GlobalCAttractorConjecture`	Open
μ-Flip Distance	`MuFlipDistanceConjecture`	Open

12 Related Work

Formalization of number theory. Mathlib [7] provides extensive number-theoretic infrastructure: divisors, primality, Fibonacci numbers, and the Mersenne gcd identity used in our proofs. The Liquid Tensor Experiment [2] and the formalization of perfectoid spaces [1] demonstrate Lean 4's capacity for large-scale mathematical formalization. The Odd Order Theorem [5] and the Kepler Conjecture [6] are landmark formalizations in Coq and HOL Light, respectively.

Prime-producing formulas. The UGP prime formula (7) is a two-parameter family constrained by the Diophantine condition (6). This places it in the tradition of prime-producing polynomials (Euler's $n^2 + n + 41$, the Bunyakovsky conjecture) and prime-generating sieves, though the constraint to ridges $2^n - 16$ is novel. The OEIS sequence A394412 [12] records the first 57 terms.

Companion work. The UGP framework was introduced in [9]; the present formalization provides the machine-checked foundation. The self-reference theorems are imported from two companion Lean 4 libraries:

- **nems-lean** [10]: Lawvere fixed-point theorem in abstract form, Kleene recursion theorem, and Rogers' fixed-point theorem, all proved from a minimal self-referential interface (SRI).
- **aps-rice-lean** [8]: Rice's theorem and halting undecidability for acceptable programming systems, proved from the s-m-n theorem and a diagonal argument.

The companion **primes-in-greedy-B3** [11] provides a related B_3 prime result: every prime appears in the greedy multiplicative B_3 sequence (OEIS A079852).

13 Conclusion

We have presented ugp-lean, a self-contained Lean 4 formalization of the Universal Generative Principle. The artifact defines the UGP framework from first principles—ridge sieve, prime-lock criterion, RSUC, GTE orbit, Quarter-Lock, Turing universality—with a strict zero-sorry, zero-custom-axiom policy on the core proof path.

The formalization contributes several results not present in prior work: the orbit-from-T derivation (theorem 5.1), the general ridge remainder lock (theorem 6.1), the even-step c-invariance theorem (theorem 5.2), the UGP prime class with ten machine-checked instances (theorem 7.3), the divisor count unboundedness theorem (theorem 10.3), the exact divisor-count formula $\tau(2^n - 16) = 5 \cdot \tau(2^{n-4} - 1)$ (theorem 10.4), and the precise formal statement of the mirror-dual conjecture (theorem 10.5) with five machine-verified witness pairs.

What remains open. The ten conjectures in table 1 are formally stated but unproved. The mirror-dual conjecture (theorem 10.5) and the UGP prime infinitude conjecture (theorem 7.5) are the most interesting: both are structurally analogous to the twin prime conjecture and would require new sieve-theoretic techniques. Dirichlet-series analysis of the ridge divisor function and Markov mixing properties of the GTE trajectory are natural directions for future formalization.

Reproducibility. The artifact is publicly available at `https://github.com/novaspivack/ugp-lean`. Building requires Lean 4 v4.29.0-rc6 and Mathlib v4.29.0-rc6. The command `lake build` compiles the entire library; no external tools beyond Lean 4 and its package manager are needed.

References

[1] Kevin Buzzard, Johan Commelin, and Patrick Massot. Formalising perfectoid spaces. In *Proc. 9th ACM SIGPLAN Int. Conf. Certified Programs and Proofs (CPP 2020)*, pages 299–312. ACM, 2020.

[2] Johan Commelin, Adam Topaz, et al. Liquid tensor experiment. `https://github.com/leanprover-community/lean-liquid`, 2022. Lean 4 formalization of Scholze's challenge.

[3] Matthew Cook. Universality in elementary cellular automata. *Complex Systems*, 15(1):1–40, 2004.

[4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Proc. 28th Int. Conf. Automated Deduction (CADE-28)*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021.

[5] Georges Gonthier, Andrea Asperti, Jeremy Avigad, Yves Bertot, Cyril Cohen, François Garillot, Stéphane Le Roux, Assia Mahboubi, Russell O'Connor, Sidi Ould Biha, Ioana Paşca, Laurence Rideau, Alexey Solovyev, Enrico Tassi, and Laurent Théry. A machine-checked proof of the odd order theorem. In *Proc. 4th Int. Conf. Interactive Theorem Proving (ITP 2013)*, volume 7998 of *LNCS*, pages 163–179. Springer, 2013.

Table 2: Complete theorem inventory. "Novel" indicates results new to this formalization; "General" indicates level-independent results. All theorems have zero `sorry` on the core proof path.

Theorem / Lean name	**Module**	**Paper ref**	**Novel?**
Core Classification (§4)			
`theoremA_general`	Classification.TheoremA	Thm A	—
`theoremB`	Classification.TheoremB	Thm B	—
`rsuc_theorem`	Classification.RSUC	RSUC	—
`mdl_selects_LeptonSeed`	Classification.TheoremB	MDL	—
`rsuc_formal`	Classification.FormalRSUC	RSUC	—
`rsuc_canon`	Classification.FormalRSUC	RSUC	—
`strengthening_cannot_add`	Classification.Monotonic	—	—
Ridge & Sieve (§4)			
`ridgeSurvivors_10`	Compute.Sieve	§5	—
`ridge_remainder_lock`	Core.RidgeRigidity	Lem m_2	—
`exclude_16` ... `exclude_63`	Compute.ExclusionFilters	§5	—
GTE Orbit (§5)			
`orbit_determined_by_T`	GTE.UpdateMap	Prop 5.1	**Yes**
`update_map_produces_orbit`	GTE.UpdateMap	Prop 5.1	**Yes**
`even_step_c_invariance`	GTE.UpdateMap	—	**Yes**
`c3_strict_eq_c2_at_n10`	GTE.UpdateMap	—	**Yes**
`gen1_mersenne_isolation`	GTE.UpdateMap	—	**Yes**
`mersenne_entanglement_gen`	GTE.UpdateMap	—	**Yes**
General-Level Theorems (§6)			
`ridge_remainder_lock_gen`	GTE.UpdateMap	Lem m_2	**General**
`quotient_gap_all`	GTE.GeneralTheorems	gap-13	**General**
`mirror_b1_invariance`	GTE.UpdateMap	—	**Yes**
`mersenne_gcd_identity`	GTE.UpdateMap	—	Mathlib
`alpha_echo`	GTE.GeneralTheorems	α-echo	**Checked**
`b_linear_growth`	GTE.GeneralTheorems	b-growth	—
`c_values_strictly_mono`	GTE.GeneralTheorems	c-mono	—
Quarter-Lock & Kernel (§8)			
`quarterLockLaw`	QuarterLock	§3	—
`quarterLockStability`	QuarterLock	§3	—
`k_L2_eq`	ElegantKernel	§3	—
`L_model_from_gauge_struct`	LModelDerivation	§3	—
Universality & Self-Reference (§9)			
`ugp_is_turing_universal`	Universality.TuringUniv	§4	—
`ugp_lawvere_fixed_point`	SelfRef.LawvereKleene	Lawvere	Import
`ugp_kleene_recursion_thm`	SelfRef.LawvereKleene	Kleene	Import
`ugp_rice_theorem`	SelfRef.RiceHalting	Rice	Import
`ugp_halting_undecidable`	SelfRef.RiceHalting	Halting	Import
UGP Primes (§7)			
`IsUGPPrime`	GTE.UGPPrimes	A394412	**Yes**
`ugp_primes_exist`	GTE.UGPPrimes	—	**Yes**
`ten_ugp_primes`	GTE.UGPPrimes	—	**Yes**
`ugp_primes_at_three_lvls`	GTE.UGPPrimes	—	**Yes**
`c1Val_eq_c1FromPair`	GTE.UGPPrimes	—	**Yes**
`UGPPrimeInfinitudeConj`	GTE.UGPPrimes	—	**Yes**
Mirror-Dual Conjecture (§10)			
`card_divisors_ridge_unb`	GTE.MirrorDualConj	—	**Yes**
`tau_ridge_exact`	GTE.MirrorDualConj	Thm 10.4	**Yes**
`coprime_pow2_mersenne`	GTE.MirrorDualConj	—	**Yes**
`five_mirror_dual_pairs`	GTE.MirrorDualConj	—	**Yes**
`MirrorDualConjecture`	GTE.MirrorDualConj	—	**Yes**

[6] Thomas Hales, Mark Adams, Gertrud Bauer, Tat Dat Dang, John Harrison, Le Truong Hoang, Cezary Kaliszyk, Victor Magron, Sean McLaughlin, Thang Tat Nguyen, et al. A formal proof of the Kepler conjecture. In *Forum of Mathematics, Pi*, volume 5, page e2. Cambridge Univ. Press, 2017.

[7] Mathlib Community. Mathlib4: The Lean 4 mathematical library. https://github.com/leanprover-community/mathlib4, 2024. Version 4.29.0-rc3.

[8] Nova Spivack. aps-rice-lean: Rice's theorem and halting undecidability for acceptable programming systems in Lean 4. https://github.com/novaspivack/aps-rice-lean, 2025.

[9] Nova Spivack. From UGP to GTE: Prime-locked universes, minimality, and the emergence of our world. Manuscript, 2025.

[10] Nova Spivack. nems-lean: Self-reference library for Lean 4 (Paper 26). https://github.com/novaspivack/nems-lean, 2025.

[11] Nova Spivack. primes-in-greedy-B3: Companion b_3 prime result in Lean 4. https://github.com/novaspivack/primes-in-greedy-B3, 2025.

[12] Nova Spivack. OEIS A394412: Primes p such that $b(q+13) = 2^n - 16$ and $p = (b+q+20)q+20$. https://oeis.org/A394412, 2026. 57 terms computed up to 10^8.